

INFORME PRÁCTICA DISEÑO

1º. Ejercicio: Seguimiento de la Flota Naval

Principios de Diseño Aplicados

Principio de Responsabilidad Única: Cada clase tiene una única responsabilidad dentro del sistema, separando las funcionalidades de acuerdo a su propósito específico:

- Clase Fleet: gestiona la flota en su totalidad, incluyendo la administración de fondos y el seguimiento de los barcos.
- Clase Ship: encapsula los datos y operaciones relacionadas con un barco específico.
- Interfaz ShipState: define los estados de un barco, asegurando que cada implementación maneje un estado específico.
- Clases de Estado: representan los diferentes estados en los que puede encontrarse un barco ("InMissionState", "InBaseState", "InactiveState", "UnderRepairState").

Principio Abierto-Cerrado: El sistema está diseñado para ser extensible sin modificar las clases existentes. Por ejemplo:

- La adición de nuevos estados de los barcos puede implementarse creando nuevas clases que extiendan la interfaz "ShipState".
- La inclusión de nuevos tipos de barcos es posible mediante la ampliación del enum "ShipType".

Principio de Sustitución de Liskov: Cada clase que implementa "ShipState" puede ser sustituida sin problemas en el sistema, ya que todas cumplen con los métodos definidos en la interfaz.

Principio de Inversión de la Dependencia: El sistema depende de abstracciones ("ShipState") en lugar de concretos, permitiendo una fácil adaptación y extensibilidad.

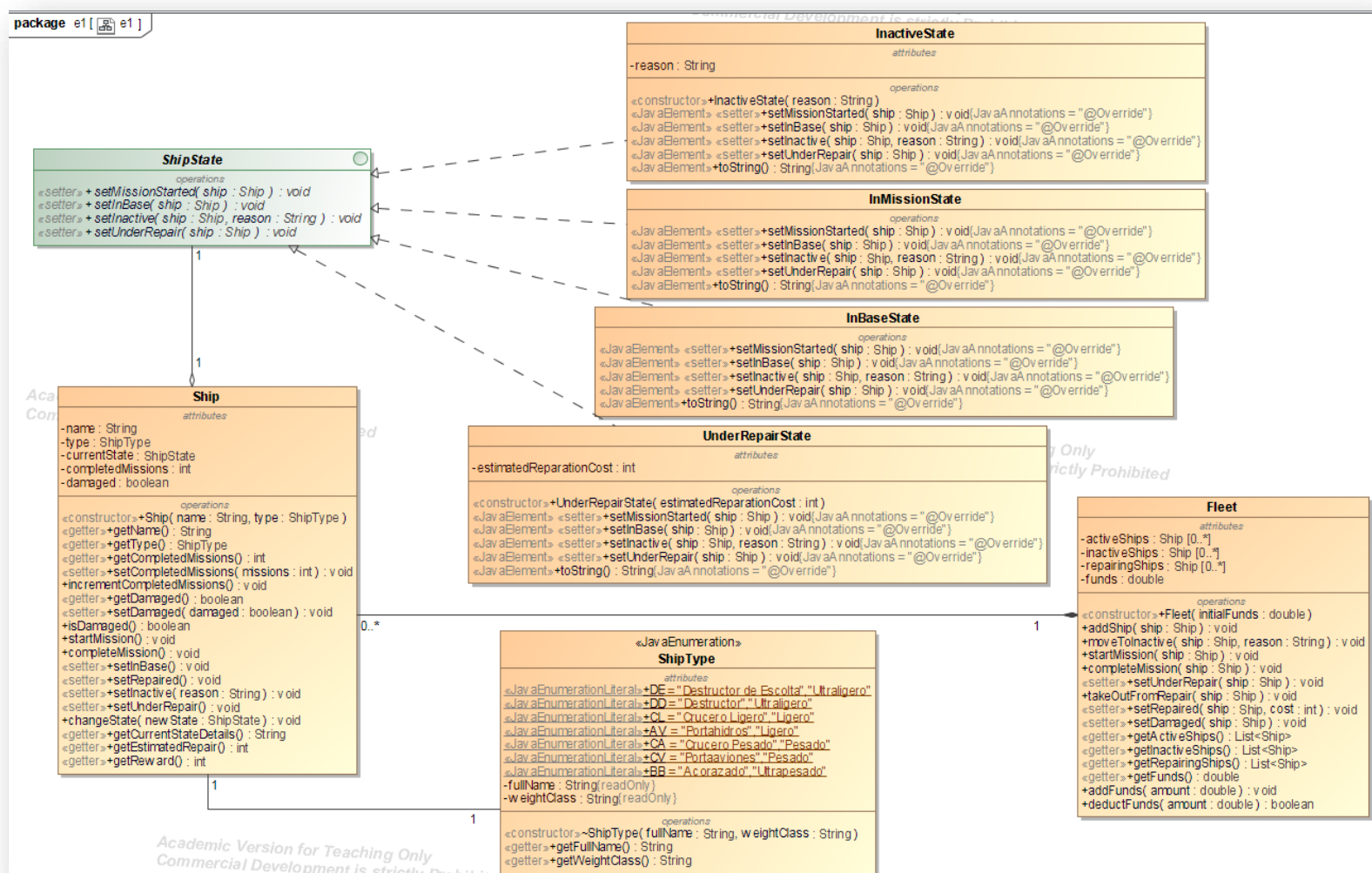
Principio de Segregación de Interfaces: Las interfaces son específicas para cada funcionalidad. Por ejemplo, "ShipState" proporciona solo los métodos necesarios para gestionar los estados de un barco.

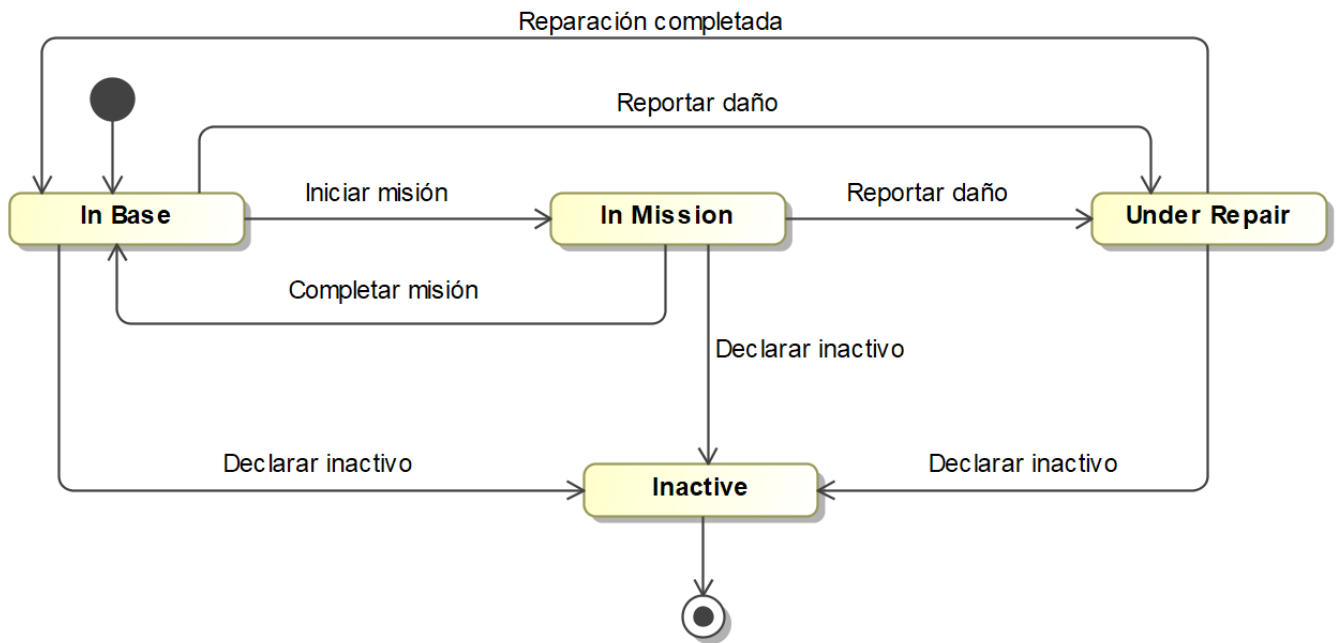
Patrones de Diseño Aplicados

Patrón Estado: El patrón estado permite que el comportamiento de un barco varíe en función de su estado actual. Las clases concretas (“InBaseState”, “InMissionState”, etc.) implementan la lógica específica de cada estado.

Patrón Estrategia: El cálculo de recompensas y costes de reparación para los barcos se delega a métodos basados en el tipo de barco (“ShipType”), encapsulando la lógica relacionada.

Patrón Composición: La clase “Fleet” gestiona listas de barcos (“activeShips”, “inactiveShips”, “repairingShips”), permitiendo operaciones flexibles y ampliables sobre estas colecciones.





2º. Ejercicio: Cotización de Acciones en el Mercado Bursátil

Principios de Diseño Aplicados

Principio de Responsabilidad Única: Cada clase tiene una única responsabilidad dentro del sistema, separando las funcionalidades de acuerdo a su propósito específico:

- Clase StockMarket: gestiona los datos de las acciones y notifica a los observadores cuando se actualizan.
- Clase StockData: encapsula la información bursátil de una acción (símbolo, precios, volumen).
- Interfaz Observer: define el contrato para los clientes interesados en recibir actualizaciones.
- Clientes (DetailedClient, SimpleTickerClient, SpecificStockClient): implementan la interfaz "Observer" para manejar diferentes tipos de actualizaciones según sus necesidades.

Principio Abierto-Cerrado: El sistema está diseñado para ser extensible sin modificar las clases existentes. Por ejemplo:

- Nuevos tipos de clientes pueden implementarse simplemente creando nuevas clases que extiendan la interfaz "Observer".
- Nuevos atributos en "StockData" pueden agregarse sin afectar la funcionalidad principal del sistema.

Principio de Sustitución de Liskov: Cada clase que implementa “Observer” puede ser sustituida sin problemas en el sistema, ya que todas cumplen con los métodos definidos en la interfaz.

Principio de Inversión de la Dependencia: El sistema depende de abstracciones (“Observer”) en lugar de concretos, permitiendo una fácil adaptación y extensibilidad.

Principio de Segregación de Interfaces: La interfaz “Observer” es específica y contiene solo los métodos necesarios para recibir actualizaciones de datos bursátiles.

Patrones de Diseño Aplicados

Patrón Observer: Este patrón permite que el “StockMarket” notifique automáticamente a los clientes interesados (“Observer”) cada vez que se actualiza la información de una acción. Las clases de clientes implementan la interfaz “Observer” para manejar las actualizaciones.

Patrón Estrategia: Cada cliente implementa su propia estrategia para procesar y mostrar la información bursátil:

- DetailedClient: muestra todos los detalles de una acción.
- SimpleTickerClient: muestra solo el precio de cierre de acciones seleccionadas.
- SpecificStockClient: muestra información detallada de una acción específica.

Patrón Composición: La clase “StockMarket” utiliza una lista de observadores para gestionar las suscripciones de los clientes, permitiendo una administración flexible y ampliable.

